

ЛАБОРАТОРНА РОБОТА №8.2

ВСТУП ДО ДОКУМЕНТАЦІЇ ПРОЄКТУ

1. ВИМОГИ

1.1. Розробник

- Студентка: Рудь Олександра
- Група: КН-925а
- Статус: студентка НТУ «Харківський політехнічний інститут»
- Дата розробки 09.11.2025

1.2. Загальне завдання

Розробити повноцінні звіти для лабораторної роботи «Функції», що присвячена реалізації та використанню функцій у мові програмування C, у двох форматах (з подальшою репрезентацією в PDF):

- Markdown (поточний файл [README.md](#));
- DOC-формат, згідно з вимогами ДСТУ, з подальшим збереженням у PDF.

При цьому необхідно:

- оформити звіт відповідно до вимог кафедри;
- створити коментарі до коду в стилі Doxygen українською мовою;
- згенерувати HTML-документацію за допомогою Doxygen;
- забезпечити узгодженість між описом у звіті та фактичною реалізацією програми.

1.3. Індивідуальне завдання

Індивідуальне завдання відсутнє.

Усі приклади та реалізації відповідають загальному завданню лабораторної роботи «Функції»

.

2. ОПИС ПРОГРАМИ

2.1. Функціональне призначення

Програма демонструє використання:

- звичайних функцій (перевірка простоти числа, робота з матрицями);
- варіативної функції (робота з довільною кількістю параметрів);
- обчислення детермінанта квадратної матриці до розміру 3×3 .

Основні можливості програми:

1. Зчитати число з аргументів командного рядка (або використати значення за замовчуванням).
2. Перевірити, чи є це число простим.
3. Продемонструвати роботу варіативної функції, що рахує кількість зростаючих пар.
4. Згенерувати випадкову квадратну матрицю 3×3 .
5. Обчислити квадрат матриці (A^2).
6. Обчислити детермінант вихідної матриці.
7. Вивести результати у консоль.

Обмеження на застосування:

- детермінант коректно обчислюється лише для квадратних матриць розміром до 3×3 ;
- вхідні параметри командного рядка мають бути цілими числами;
- максимальна кількість аргументів для варіативної функції обмежена розміром внутрішнього масиву (1000 елементів).

2.2. Опис логічної структури

2.2.2. Опис основних функцій

У програмі реалізовано такі основні функції:

1. `bool isPrime(int number);`

- Призначення: перевіряє, чи є ціле число простим.
- Вхідні дані:

■ `number` — ціле число, яке перевіряється.

- Результат:
 - `true`, якщо число просте (ділиться лише на 1 і на себе);
 - `false`, якщо число має інші дільники або `number <= 1`.

2. `int random_int(int minimum, int maximum);`

- Призначення: генерує випадкове ціле число в діапазоні `[minimum; maximum]`.
- Вхідні дані:
 - `minimum` — нижня межа (включно);
 - `maximum` — верхня межа (включно).
- Результат: випадкове ціле число в заданому діапазоні.
- Особливості: для отримання різних послідовностей чисел використовується попередня ініціалізація генератора випадкових чисел через `srand(time(NULL));`.

3. `void printMatrix(int matrix_array[], int rows, int cols);`

- Призначення: виводить матрицю на екран у вигляді таблиці.
- Вхідні дані:
 - `matrix_array` — одновимірний масив, у якому матриця збережена у форматі row-major;
 - `rows` — кількість рядків;
 - `cols` — кількість стовпців.
- Результат: виведення елементів матриці у вигляді таблиці з табуляцією між стовпцями.

4. `void squareMatrix(int matrix_array[], int rows, int cols, int result[]);`

- Призначення: обчислює квадрат квадратної матриці A (тобто A^2).
- Вхідні дані:
 - `matrix_array` — вхідна матриця A (одновимірний масив);
 - `rows` — кількість рядків;
 - `cols` — кількість стовпців;
 - `result` — масив для збереження результату множення.
- Результат: у масив `result` записується матриця A^2 .
- Обмеження: передбачено використання квадратних матриць (`rows = cols`).

5. `int countIncreasingPairs(int count, ...);`

- Призначення: варіативна функція, яка рахує кількість зростаючих пар (i, k) , для яких `array[i] < array[k]` при $i < k$.
- Вхідні дані:
 - `count` — кількість подальших цілих аргументів;
 - `...` — змінна кількість цілих чисел.
- Результат: ціле число — кількість зростаючих пар.
- Особливості: усі аргументи зчитуються у локальний масив `array_count`, після чого перебираються всі можливі пари.

6. `int get_determinant(int matrix[], int rows, int cols);`

- Призначення: обчислює детермінант квадратної матриці розміром до 3×3 .
- Вхідні дані:
 - `matrix` — одновимірний масив елементів матриці;
 - `rows` — кількість рядків;
 - `cols` — кількість стовпців.
- Результат: значення детермінанта.
- Особливості:
 - для 1×1 детермінант дорівнює єдиному елементу;
 - для 2×2 використовується формула `ad - bc`;
 - для 3×3 використовується формула Саррюса;
 - якщо матриця не квадратна або її розмір більший за 3×3 — повертається 0.

7. `int main(int argc, char* argv[]);`

- Призначення: точка входу в програму, демонструє послідовний виклик усіх описаних вище функцій.
- Основні дії:
 - зчитування числа з аргументів командного рядка (або використання значення 6 за замовчуванням);
 - перевірка числа на простоту з використанням `isPrime`;
 - демонстрація роботи варіативної функції `countIncreasingPairs`;

- створення та заповнення випадковими значеннями квадратної матриці 3×3 ;
- обчислення A^2 за допомогою `squareMatrix`;
- виведення вихідної та квадратної матриць;
- обчислення детермінанта вихідної матриці через `get_determinant`.

2.2.3. Структура даних

У програмі використовуються такі основні структури даних:

- Двовимірний масив `int matrix[n][n]`; — для зручного заповнення матриці випадковими числами.
- Одновимірні масиви `matrix_array[n * n]` та `result[n * n]` — для зберігання вихідної матриці та результату множення у форматі row-major.
- Одновимірний масив `array_count[1000]`; у функції `countIncreasingPairs` для тимчасового збереження аргументів варіативної функції.

2.3. Важливі фрагменти програми:

```
bool isPrime (int number){
    if (number <= 1) {
        return false; // числа <=1 не є простими
    }

    for (int i = 2; i < number; i++) {
        if (number % i == 0) {
            return false; // якщо знайдено дільник - число не просте
        }
    }

    return true; // якщо не знайдено дільників - число просте
}
```

Рис. 1. Перевірка числа на простоту

Опис: функція послідовно перевіряє всі можливі дільники числа від 2 до **number - 1**. Якщо знайдено хоча б один дільник без остачі — число не є простим.

```
int countIncreasingPairs(int count, ...){
    va_list args;
    va_start(args, count);

    int pairs_count = 0;
    int array_count[1000];

    // Зчитуємо передані аргументи у масив
    for(int i = 0; i < count; i++){
        array_count[i] = va_arg(args, int);
    }

    // Перебираємо всі можливі пари
    for(int i = 0; i < count; i++){
        for(int k = i + 1; k < count; k++){
            if(array_count[i] < array_count[k]){
                pairs_count++; // збільшуємо лічильник, якщо пара зростаюча
            }
        }
    }

    va_end(args);
    return pairs_count;
}
```

Рис 2. Варіативна функція для підрахунку зростаючих пар

Опис: значення переданих аргументів зберігаються у масиві `array_count`. Далі перебираються всі пари $i < k$, де порівнюються відповідні елементи. Якщо `array_count[i] < array_count[k]`, лічильник зростаючих пар збільшується на одиницю.

```

int get_determinant(int matrix[], int rows, int cols){
    if (rows != cols){
        return 0; // детермінант лише для квадратних матриць
    }

    if (rows == 1) {
        return matrix[0]; // для 1x1 детермінант - сам елемент
    }

    if (rows == 2) {
        // |a b|
        // |c d| = ad - bc
        return matrix[0] * matrix[3] - matrix[1] * matrix[2];
    }

    if (rows == 3) {
        // |a b c|
        // |d e f|
        // |g h i|
        int a = matrix[0];
        int b = matrix[1];
        int c = matrix[2];
        int d = matrix[3];
        int e = matrix[4];
        int f = matrix[5];
        int g = matrix[6];
        int h = matrix[7];
        int i = matrix[8];

        // Формула Саррюса для 3x3
        int det = a * (e * i - f * h)
                 - b * (d * i - f * g)
                 + c * (d * h - e * g);

        return det;
    }

    return 0; // для більших матриць не реалізовано
}

```

Рис 3. Обчислення детермінанта 3×3

Опис: для матриці 3×3 використовується класична формула Саррюса. Для розмірностей, більших за 3×3, функція повертає 0, що є обмеженням реалізації.

3. ВАРІАНТИ ВИКОРИСТАННЯ

3.1. Опис взаємодії з програмою

Програма запускається з командного рядка. Користувач може:

- не передавати додаткових аргументів — тоді за замовчуванням буде використано число 6;
- передати одне ціле число як аргумент — воно буде використано для перевірки на простоту.

Хід роботи програми:

1. Зчитується число з аргументів `argv` або встановлюється значення 6.
2. Це число перевіряється на простоту за допомогою функції `isPrime`.
3. Результат перевірки виводиться на екран у вигляді повідомлення:
 - `X is prime` або
 - `X is not prime`.
4. Демонструється робота варіативної функції `countIncreasingPairs` на фіксованому наборі аргументів (у коді — 5, 5, 4, 3, 2, 1).
5. Генерується випадкова квадратна матриця розміром 3×3 з елементами у діапазоні від 0 до 100.
6. Матриця копіюється у одновимірний масив та множиться сама на себе (`squareMatrix`).
7. На екран виводяться:
 - вихідна матриця (`source matrix`);
 - квадрат матриці (`squared matrix`).
8. Обчислюється та виводиться детермінант вихідної матриці.

3.2. Приклад запуску програми

```
./dist/main.bin
6 is not prime
pairs_count: 0
source matrix:
6      12      6
59     67      7
29     18     50
squared matrix:
918    984    420
4510   5323   1173
2686   2454   2800
determinant of source matrix: -18906
```

4. ВИСНОВКИ

У ході виконання лабораторної роботи № 8.2 «Вступ до документації проекту» було:

- реалізовано програму мовою C, яка демонструє роботу з функціями, у тому числі варіативними;

- виконано перевірку числа на простоту, підрахунок зростаючих пар у послідовності, операції над квадратною матрицею 3×3 та обчислення її детермінанта;
- продемонстровано використання генератора випадкових чисел `rand()` та його ініціалізації через `srand(time(NULL))`;
- закріплено навички роботи з масивами, матрицями, циклами та умовними операторами;
- опрацьовано вимоги до оформлення звітів у форматах doc (згідно ДСТУ), Markdown та їх подальшої PDF-репрезентації;
- сформовано структуру звіту з розділами «Вимоги», «Опис програми», «Варіанти використання», «Висновки».

Мета лабораторної роботи досягнута: програма функціонує коректно, а звіт оформлений відповідно до поставлених вимог щодо структури та змісту документації проєкту.